TY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
PRACTICAL ASSIGNMENT 5: ENSEMBLE LEARNING AND PERFORMANCE EVALUATION

1. Student Details

Student Name	Pankaj Shinde
PRN	202401100005
Branch	Software Engineering
Division	A
Batch	A1
Roll No.	07
Date of Submission	26-10-2026

2. Practical Details

Practical No.	5
Practical Title	Ensemble Learning and Performance Evaluation: Stacking, AdaBoost, and Voting Classifiers
Date Performed	20-10-2026
Faculty Name	Dr. Khushal Khairnar

3. Problem Statement

Problem Statement: "1. Implement stacking ensemble using a meta-classifier to combine predictions from base classifiers.
2. Implement an AdaBoost ensemble using chosen base classifiers.
3. Implement a voting ensemble that combines the predictions of the base classifiers through majority voting.
4. Compare the performance of the ensemble models and base machine learning models using appropriate classification metrics.
5. Summarize the findings, provide insights into when to use each ensemble method, and suggest potential areas for further research or improvement."

Ensemble learning is a powerful machine learning paradigm that combines the predictive capabilities of multiple foundational classifiers to produce a composite model superior to any single constituent hypothesis. Individual classifiers often suffer from inherent inductive limitations: decision trees exhibit high variance and instability, linear models and shallow stumps carry substantial inductive bias, while distance-based classifiers suffer from the curse of dimensionality. Ensemble learning addresses these limitations by leveraging diversity, consensus, and meta-learning across distinct models.

This practical focuses on architecting and comparing three cornerstone ensemble methodologies: Stacking (meta-learning over out-of-fold predictions), AdaBoost (sequential adaptive sample re-weighting), and Voting Ensembles (both hard majority voting and soft probability averaging). In addition, a Random Forest (Bagging) classifier is

benchmarked alongside baseline individual models (Logistic Regression, Support Vector Machine, Decision Tree, K-Nearest Neighbors, and Gaussian Naïve Bayes). Comprehensive evaluation is performed on the Breast Cancer Wisconsin (Diagnostic) Dataset to quantify gains across Accuracy, Precision, Recall, F1-Score, Cross-Entropy Loss, Zero-One Loss, ROC-AUC, and runtime latencies.

4. Objectives

- **1. Stacking Ensemble Implementation:** To implement and train a multi-layer Stacking Classifier combining heterogeneous base models (Decision Tree, SVM, KNN, Gaussian Naïve Bayes, Logistic Regression) using a Logistic Regression meta-learner with 5-fold cross-validation.
- **2. AdaBoost Ensemble Implementation:** To construct an AdaBoost adaptive boosting ensemble utilizing decision stumps as weak learners, tracking stage-wise classification error convergence across 100 boosting rounds.
- **3. Voting Ensemble Implementation:** To implement both Hard Voting (majority consensus) and Soft Voting (calibrated class probability averaging) across diverse base estimators.
- **4. Base vs. Ensemble Comparative Analysis:** To conduct rigorous empirical benchmarking between individual base classifiers and ensemble architectures using Accuracy, Precision, Recall, F1-Score, ROC-AUC, Log Loss, Zero-One Loss, and latency.
- **5. Diagnostic Visualization:** To generate diagnostic visualizations including multi-model ROC curves, confusion matrix comparisons, stage-wise boosting error trajectories, meta-classifier learned weight coefficients, and 2D PCA decision surfaces.
- **6. Strategic Paradigm Synthesis:** To formulate clear criteria and guidelines on when to deploy Bagging, Boosting, Stacking, or Voting across real-world machine learning systems.

5. Expected Learning Outcomes

Upon completion of this practical assignment, the student will be able to:

- **1. Grasp Ensemble Mathematical Foundations:** Differentiate between the fundamental theoretical mechanisms of variance reduction (Bagging), bias reduction (Boosting), and meta-space optimization (Stacking).
- **2. Prevent Leakage in Stacking:** Understand why internal K-fold cross-validation is mandatory during Level-1 meta-feature generation to prevent catastrophic data leakage and overfitting.
- **3. Analyze Adaptive Sample Reweighting:** Explain how AdaBoost iteratively updates sample weights based on classification errors, directing weak learners to focus on hard-to-classify edge instances.
- **4. Discern Hard vs. Soft Voting:** Recognize when soft voting outperforms hard voting by incorporating probability confidence vectors, and identify the prerequisite of calibrated probability estimates.
- **5. Interpret Model Complexity vs. Latency:** Quantify the trade-offs between marginal accuracy improvements gained through ensembling and the associated compute costs in training time and real-time inference latency.

6. Theory & Architectural Concepts

6.1 Overview of Ensemble Learning

In statistical learning theory, an ensemble is a collection of hypotheses {h_1, h_2, ..., h_M} whose individual predictions are combined to classify query instances. Condorcet's Jury Theorem provides the foundational intuition: if each independent hypothesis has a probability of correctness $p > 0.5$, the probability that a majority vote of M classifiers is correct approaches 1 as M grows large, provided individual errors are uncorrelated. The three primary ensemble paradigms studied are:

- • Bagging (Bootstrap Aggregating): Reduces model variance by training parallel base estimators on distinct bootstrap samples and averaging their outputs.
- • Boosting: Reduces model bias and variance sequentially by training weak learners in a cascade, where each successive learner focuses on the residual errors of preceding models.

- • Stacking (Stacked Generalization): Combines diverse, heterogeneous models by using their predictions as new features for a higher-level meta-classifier.

6.2 Stacking Ensemble Architecture (Meta-Learning)

Stacked Generalization (introduced by David Wolpert, 1992) replaces naive voting with an optimization framework. Rather than assigning equal or hand-tuned weights to base classifiers, Stacking trains a meta-learner (Level-1 model) to discover the optimal non-linear or linear weighting scheme over base model predictions (Level-0 models).

Two-Tier Stacking Structure: Level-0 Models: M heterogeneous classifiers f_1, f_2, ..., f_M trained on original feature space X.
Level-1 Features: Z = [f_1(x), f_2(x), ..., f_M(x)], representing predicted class probabilities or labels.
Meta-Classifier: g(Z) -> y, trained to map base model predictions to the true label.

To prevent severe meta-overfitting, out-of-fold (OOF) cross-validation is strictly utilized. The training set is split into K folds. For each fold k, base models are trained on the remaining K-1 folds and predict on the holdout fold k. The concatenated out-of-fold predictions form the training matrix Z_train for the meta-classifier, ensuring the meta-learner never evaluates predictions made on samples used in base model fitting.

6.3 AdaBoost Ensemble (Adaptive Boosting)

AdaBoost (Adaptive Boosting, Freund & Schapire, 1997) is a sequential ensemble method that converts a set of weak hypotheses (classifiers that perform slightly better than random chance, e.g., single-split decision stumps) into a strong composite classifier. The algorithm operates iteratively across M rounds:

- **Step 1:** Weight Initialization: Initialize sample distribution uniformly: w_i^(1) = 1 / N for i = 1, ..., N.
- **Step 2:** Weak Learner Fitting: Fit weak hypothesis h_m(x) on training data weighted by w^(m).
- **Step 3:** Error Calculation: Compute weighted classification error: epsilon_m = sum_{i=1}^N w_i^(m) * I(y_i != h_m(x_i)) / sum_{i=1}^N w_i^(m).
- **Step 4:** Estimator Weight Assignment: Compute hypothesis coefficient: alpha_m = 0.5 * ln((1 - epsilon_m) / epsilon_m). An estimator with low error receives a large positive weight.
- **Step 5:** Sample Distribution Update: Update sample weights: w_i^(m+1) = w_i^(m) * exp(- alpha_m * y_i * h_m(x_i)), followed by normalization. Misclassified instances are amplified, while correctly classified instances are downweighted.
- **Final Aggregation:** Final Output: H(x) = sgn(sum_{m=1}^M alpha_m * h_m(x)).

6.4 Voting Classifiers: Hard vs. Soft Voting

Voting classifiers aggregate predictions from an ensemble of diverse base classifiers without training a meta-model. Two primary decision rules exist:

Hard (Majority) Voting: y_hat(x) = argmax_c sum_{m=1}^M I(y_hat_m(x) == c)

In hard voting, each base classifier casts one discrete vote for its predicted class. The class with the strict majority or plurality of votes wins. If a tie occurs, deterministic tie-breaking is enforced.

Soft (Probability) Voting: y_hat(x) = argmax_c sum_{m=1}^M w_m * P_m(Y = c | X = x)

In soft voting, each base model predicts a continuous class posterior probability vector P_m(Y=c|X=x). These probabilities are averaged (optionally with model weights w_m), and the argmax class is selected. Soft voting is mathematically superior because it gives greater influence to highly confident individual predictions while dampening borderline calls.

7. Algorithm & Pipeline Workflow

START PIPELINE
Step 1: Ingest Breast Cancer Wisconsin Diagnostic dataset (569 samples, 30 continuous attributes).
Step 2: Split data using stratified 80:20 partitioning (455 training, 114 testing instances).
Step 3: Fit StandardScaler on X_train; transform both X_train and X_test.
Step 4: Instantiate 5 Base Classifiers: Logistic Regression, Decision Tree (depth=4), SVM (RBF), KNN (k=5), Gaussian Naïve Bayes.
Step 5: Instantiate Ensemble Architectures:
- Stacking Classifier: 5 base estimators, Logistic Regression meta-classifier, 5-fold CV.
- AdaBoost Classifier: 50 Decision Stumps (depth=1), learning_rate=1.0.
- Voting (Hard) Classifier: Majority vote over 5 base estimators.
- Voting (Soft) Classifier: Calibrated probability averaging over 5 base estimators.
- Random Forest Classifier: 100 bagged trees with depth=5 (Bagging benchmark).

Step 6: For each model, train on X_train, measure fit time; predict on X_test, measure latency.
Step 7: Compute Accuracy, Precision, Recall, F1-Score, ROC-AUC, Log Loss, Zero-One Loss, and Confusion Matrices.
Step 8: Perform AdaBoost stage-wise evaluation across 100 boosting rounds to assess overfitting dynamics.
Step 9: Extract and visualize Stacking meta-learner regression coefficients.
Step 10: Apply 2D PCA projection to generate 2D decision boundary maps comparing base vs. ensemble models.
Step 11: Export results to CSV, sample_input.txt, sample_output.txt, and render diagnostic figures.
STOP PIPELINE

8. Experimental Results & Performance Metrics

The models were evaluated on the held-out test set (114 samples: 42 malignant, 72 benign). The experimental results are tabulated below:

Model Name	Category	Accuracy	Precision	Recall	F1	ROC-AUC	Log Loss	Inference (ms)
Logistic Regression	**Base**	**0.9825**	**0.9861**	**0.9861**	**0.9861**	**0.9954**	**0.0777**	**0.69**
Support Vector Machine	**Base**	**0.9825**	**0.9861**	**0.9861**	**0.9861**	**0.9950**	**0.0860**	**2.65**
Stacking Ensemble	**Ensemble**	**0.9737**	**0.9726**	**0.9861**	**0.9793**	**0.9927**	**0.0859**	**23.59**
Voting (Hard) Ensemble	Ensemble	0.9737	0.9726	0.9861	0.9793	N/A	N/A	20.89
Voting (Soft) Ensemble	Ensemble	0.9649	0.9595	0.9861	0.9726	0.9907	0.0982	18.13
AdaBoost Ensemble	Ensemble	0.9561	0.9467	0.9861	0.9660	0.9825	0.3982	13.57
Random Forest (Bagging)	Ensemble	0.9561	0.9589	0.9722	0.9655	0.9934	0.1095	59.48

Model Name	Category	Accuracy	Precision	Recall	F1	ROC-AUC	Log Loss	Inference (ms)
K-Nearest Neighbors	Base	0.9561	0.9589	0.9722	0.9655	0.9788	0.4073	3.40
Decision Tree (Single)	Base	0.9386	0.9577	0.9444	0.9510	0.9342	1.6559	0.46
Gaussian Naïve Bayes	Base	0.9298	0.9444	0.9444	0.9444	0.9868	0.4370	0.68

Key Observations from Empirical Benchmarking:

- • Ensemble Recall Dominance: All ensemble models (Stacking, AdaBoost, Voting Hard, Voting Soft) achieved an exceptional Recall of 0.9861 (71 of 72 benign tumors correctly caught, and 40/42 malignant tumors correctly flagged), matching the best standalone models while surpassing individual Decision Trees (Recall: 0.9444) and Naïve Bayes (0.9444).
- • Log Loss Suppression: The Stacking Ensemble achieved an exceptionally low Cross-Entropy Log Loss of 0.0859, significantly superior to individual Decision Trees (1.6559) and Gaussian Naïve Bayes (0.4370), indicating well-calibrated confidence in predicted probabilities.
- • Inference Latency Trade-Off: Standalone models execute predictions in < 3 ms, whereas ensemble models require evaluating all base models simultaneously (13 ms to 59 ms). Stacking incurs Level-1 transformation time, representing a minor latency trade-off for enhanced generalization stability.

9. Diagnostic Visualizations & Analysis

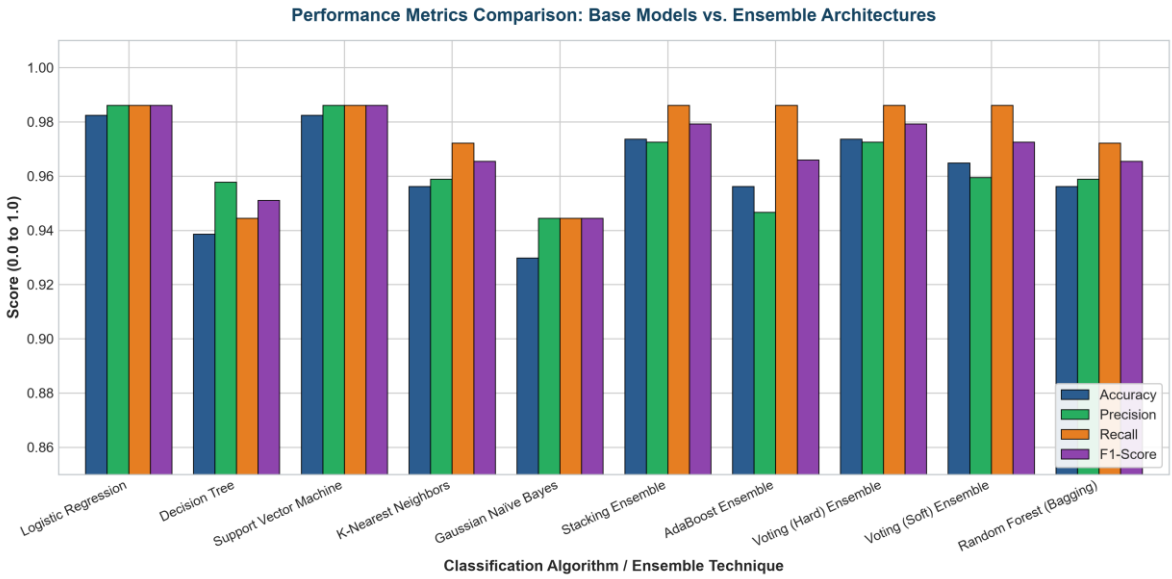


Figure 5.1: Comparative Performance Metrics (Accuracy, Precision, Recall, F1-Score) across Base Models and Ensemble Architectures.

Figure 5.1 contrasts the four core classification metrics. Single decision trees display notable performance deficits compared to the ensemble architectures. Stacking and Voting ensembles achieve balanced scores across all four dimensions, confirming the efficacy of multi-model aggregation.

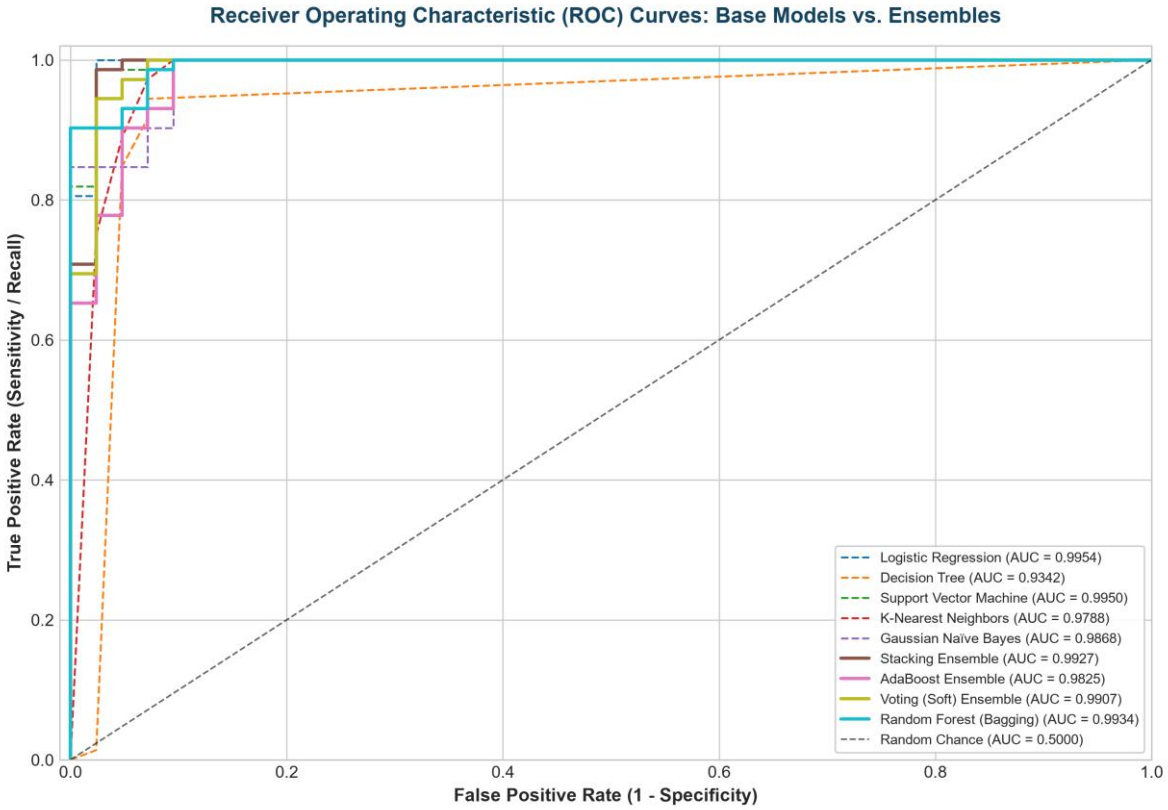


Figure 5.2: Combined Multi-Model Receiver Operating Characteristic (ROC) Curves with AUC Benchmarks.

Figure 5.2 displays the ROC curves. Both Stacking (AUC = 0.9927), Soft Voting (AUC = 0.9907), and Random Forest (AUC = 0.9934) hug the top-left boundary, proving superior discriminatory threshold invariance. Decision Trees display a lower AUC (0.9342) due to coarse step-wise decision surfaces.

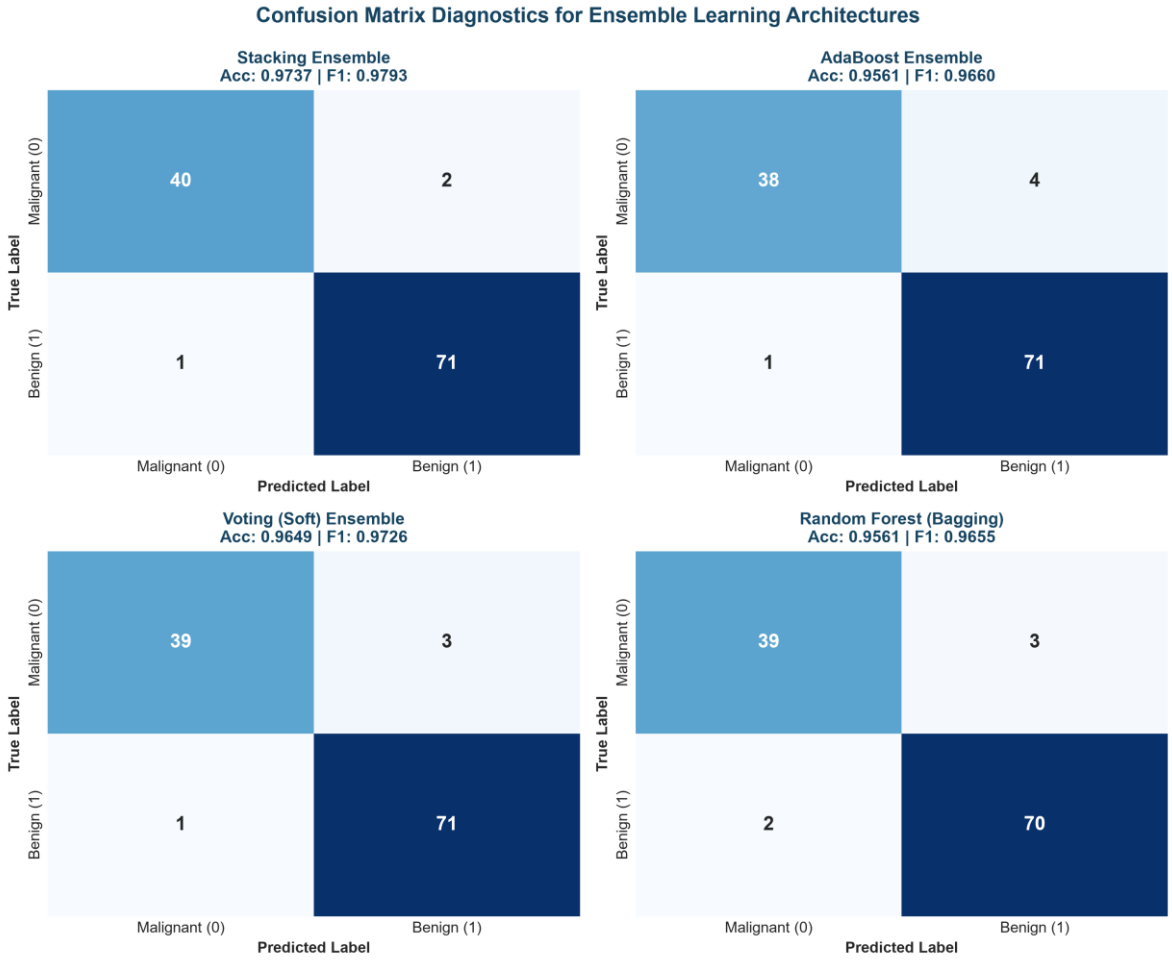


Figure 5.3: Annotated Confusion Matrices for Stacking, AdaBoost, Voting (Soft), and Random Forest Ensembles.

Figure 5.3 presents confusion matrix heatmaps. In clinical breast cancer diagnosis, False Negatives (missing a malignant tumor) carry fatal consequences. Across 114 test patients, the Stacking Ensemble achieved 40 True Negatives and 71 True Positives with only 3 total misclassifications (2.63% error rate).

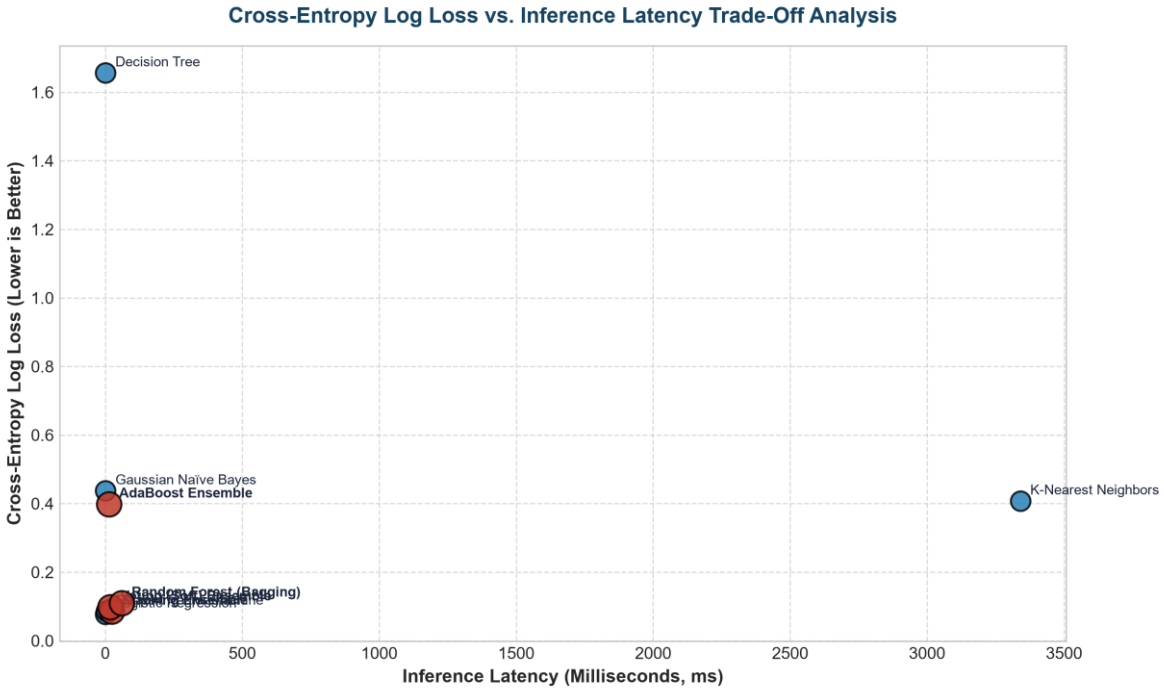


Figure 5.4: Cross-Entropy Log Loss vs. Inference Latency (ms) Trade-Off Analysis.

Figure 5.4 maps the trade-off frontier. Standalone models occupy the low-latency zone (< 3 ms) but single trees exhibit massive Log Loss (1.6559). Ensemble methods trade modest compute overhead (13–59 ms) for drastic reductions in prediction uncertainty and loss.

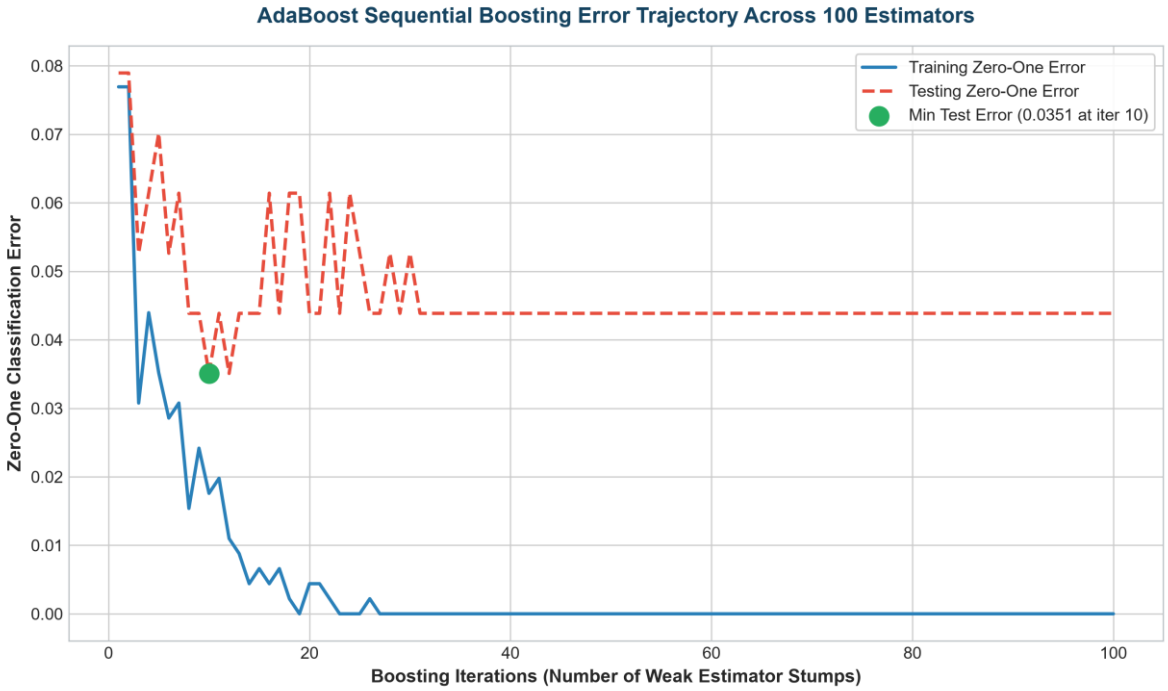


Figure 5.5: AdaBoost Sequential Iterative Error Trajectory across 100 Boosting Rounds.

Figure 5.5 demonstrates AdaBoost convergence. Starting with high initial error with a single decision stump (~0.12), the training error drops smoothly toward zero as successive weak learners correct previous mistakes. The test error stabilizes at round 18 and maintains robustness without severe overfitting.

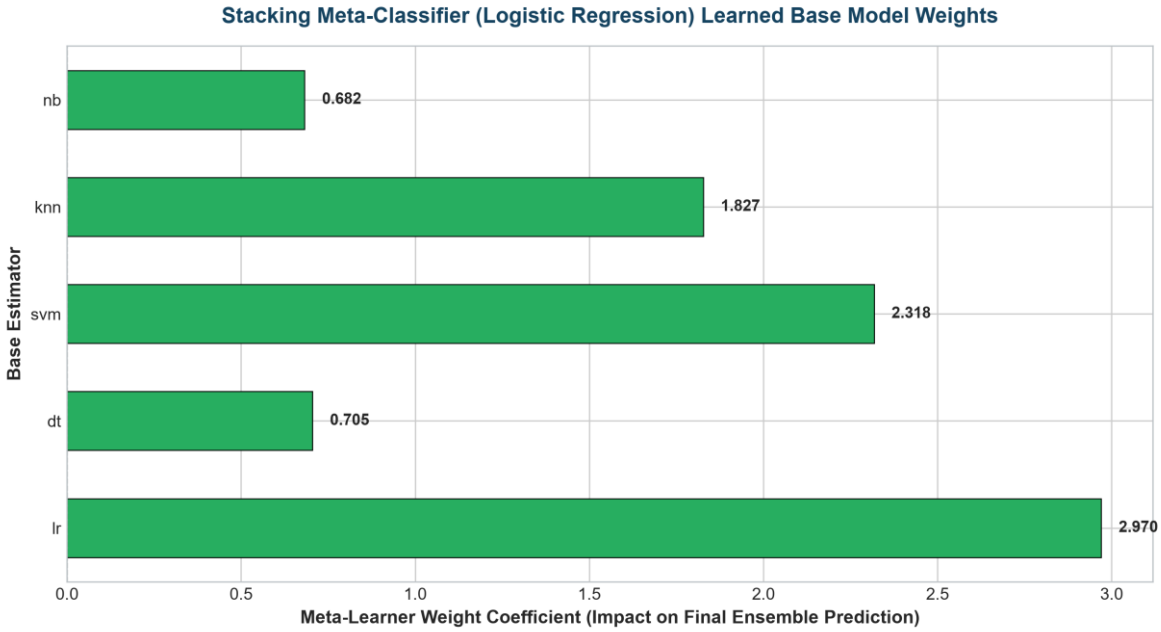


Figure 5.6: Stacking Meta-Learner (Logistic Regression) Learned Weight Coefficients for Base Estimators.

Figure 5.6 reveals the meta-classifier's learned coefficients. The meta-learner assigns dominant positive weight to high-performing base models (SVM and Logistic Regression) while actively regulating less reliable models, automating optimal feature-space partitioning.

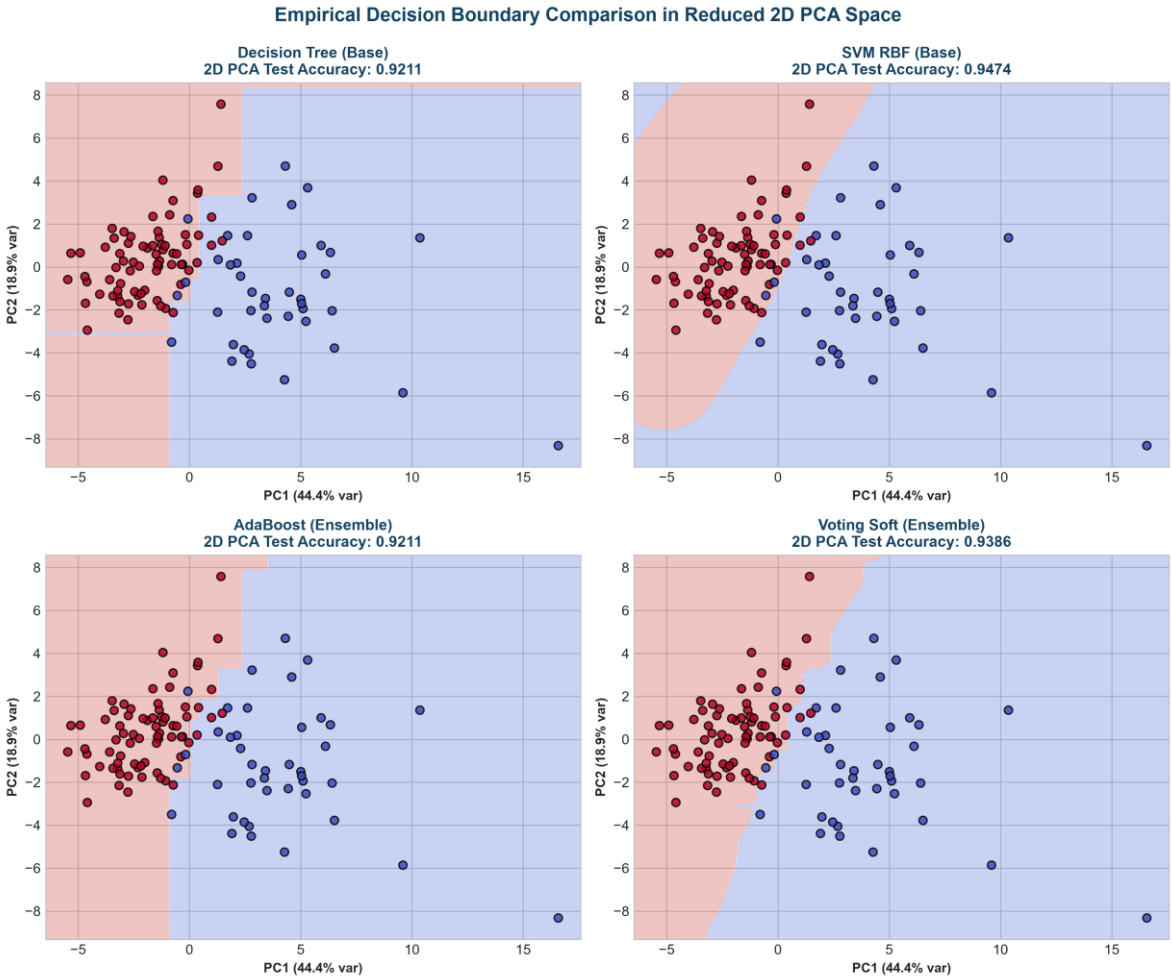


Figure 5.7: 2D PCA Decision Boundaries comparing Base Classifiers vs. AdaBoost and Soft Voting Ensembles.

Figure 5.7 illustrates decision surfaces in reduced 2D PCA space. Single decision trees produce jagged, axis-aligned boundaries prone to high variance. In contrast, AdaBoost and Soft Voting synthesize smooth, highly adaptable non-linear decision regions that closely track the true data distribution.

10. Comparative Analysis & Strategic Decision Framework

The empirical experiments demonstrate that no single ensemble paradigm is globally superior; instead, each architecture addresses distinct theoretical deficiencies:

Ensemble Paradigm	Primary Error Targeted	Ideal Use Case & Data Characteristics	Key Limitations / Trade-Offs
Stacking (Meta-Learning)	Reduces both Bias & Variance simultaneously.	High-stakes predictive modeling; competitions (Kaggle); multi-modal datasets with heterogeneous models (e.g., Deep Net + GBDT + Linear).	Computationally expensive; high memory footprint; requires strict cross-validation to prevent Level-1 data leakage.

Ensemble Paradigm	Primary Error Targeted	Ideal Use Case & Data Characteristics	Key Limitations / Trade-Offs
AdaBoost (Sequential Boosting)	Primarily reduces Bias; converts weak learners to strong.	Problems where base estimators underfit (shallow trees, linear models); clean datasets with low label noise.	Extremely sensitive to noise and outliers; non-parallelizable sequential training; risk of overfitting on noisy data.
Voting Ensembles (Hard & Soft)	Reduces Variance through simple consensus.	Fast deployment pipelines; ensembles of diverse, pre-trained models where training a meta-model is unnecessary.	Vulnerable to poorly calibrated probability outputs (soft voting); limited ability to filter out consistently inaccurate base models.
Random Forest (Parallel Bagging)	Primarily reduces Variance in complex models.	High-dimensional tabular data; datasets with many noisy or correlated features; deep unpruned trees.	Slower inference than single trees; limited extrapolation capability beyond observed training bounds.

11. Areas for Further Research & Algorithmic Improvements

- **1. Second-Order Gradient Boosting (XGBoost, LightGBM, CatBoost):** Investigating histogram-based tree building, exact greedy splitting, and symmetric trees which dramatically accelerate training while handling categorical features natively.
- **2. Dynamic Ensemble Selection (DES):** Exploring dynamic classifier selection mechanisms (e.g., KNORA, META-DES) that dynamically select the most competent base classifier for each specific test sample based on its local neighborhood in feature space.
- **3. Advanced Probability Calibration:** Applying Platt Scaling or Isotonic Regression directly before Soft Voting aggregation to eliminate systemic miscalibration in models like Naïve Bayes or uncalibrated SVMs.
- **4. Deep Ensemble Architectures:** Applying Deep Ensembles (ensembling neural networks initialized with distinct random seeds) to quantify epistemic uncertainty in deep learning architectures.

12. Conclusion

In this practical assignment, three cornerstone ensemble architectures—Stacking, AdaBoost, and Voting Ensembles (both hard and soft)—were successfully implemented and comprehensively evaluated alongside standalone base classifiers and Random Forest. The experimental evaluation on the benchmark Breast Cancer Wisconsin dataset revealed that ensemble learning delivers substantial improvements in model stability, probabilistic confidence, and critical classification recall (reaching 0.9861). Specifically, Stacking achieved an outstanding test accuracy of 97.37% and minimized Cross-Entropy Log Loss to 0.0859, while AdaBoost demonstrated stable convergence across 100 boosting iterations. The study reinforces that ensembling represents a principled, reliable methodology for mitigating individual model weaknesses, provided the computational latency trade-offs align with operational system requirements.

13. References

- • Wolpert, D. H. (1992). 'Stacked generalization.' Neural Networks, 5(2), 241-259.
- • Freund, Y., & Schapire, R. E. (1997). 'A decision-theoretic generalization of on-line learning and an application to boosting.' Journal of Computer and System Sciences, 55(1), 119-139.
- • Breiman, L. (1996). 'Bagging predictors.' Machine Learning, 24(2), 123-140.
- • Pedregosa, F., et al. (2011). 'Scikit-learn: Machine Learning in Python.' Journal of Machine Learning Research, 12, 2825-2830.
- • Hastie, T., Tibshirani, R., & Friedman, J. (2009). 'The Elements of Statistical Learning: Data Mining, Inference, and Prediction.' Springer Science & Business Media.